Installing Magic, Netgen, IRSIM, ngspice, xschem, and open_pdks

An installation guide for the Open Circuit Design tool suite

James E. Stine and Landon Burleson

April 30, 2026

1 Overview

This guide walks through building and installing the open-source analog/digital IC design flow maintained primarily by Tim Edwards into a self-contained user prefix. The default location used throughout this document is

```
$HOME/opencircuitdesign
```

which matches the default `OPEN_CIRCUITDESIGN_ROOT` value in the included `setup.sh`. If you would rather install elsewhere (e.g. `/opt/opencircuitdesign` or `/home/user/ocd`), simply edit the line near the top of `setup.sh`:

```
export OPEN_CIRCUITDESIGN_ROOT="$HOME/opencircuitdesign"
```

and use that same path everywhere this guide writes `$OPEN_CIRCUITDESIGN_ROOT`.

The tools covered are:

- **Magic** — VLSI layout editor (http://opencircuitdesign.com/magic).
- **Netgen** — netlist comparison / LVS (http://opencircuitdesign.com/netgen).
- **IRSIM** — switch-level digital simulator (http://opencircuitdesign.com/irsim).
- **ngspice** — analog/mixed-signal SPICE simulator (https://ngspice.sourceforge.io). *Note:* ngspice is a separate project from Tim Edwards' tools, but is required by `open_pdks` and is conventionally bundled with the suite.
- **xschem** — schematic capture (https://github.com/StefanSchippers/xschem). Also a separate upstream, included because `setup.sh` configures it.
- **open_pdks** — installer/wrapper for open-source PDKs such as `sky130A` and `gf180mcu` (https://github.com/RTimothyEdwards/open_pdks).

Everything will be installed under a single prefix:

```
$OPEN_CIRCUITDESIGN_ROOT
|-- bin/        # magic, netgen, irsim, ngspice, xschem executables
|-- lib/        # shared libraries
|-- share/      # PDKs, scripts, technology files, magic/netgen/xschem libs
|-- include/    # development headers
```

2 Prerequisites

2.1 Build dependencies (Debian / Ubuntu)

Install the toolchain and X/Tcl libraries before building any of the tools.

```
sudo apt-get update
sudo apt-get install -y \
    build-essential m4 tcsh csh git \
    tcl-dev tk-dev \
    libcairo2-dev \
    mesa-common-dev libglu1-mesa-dev \
    libncurses-dev \
    libx11-dev libxpm-dev libxt-dev \
    libreadline-dev \
    bison flex \
    autoconf automake libtool \
    libfftw3-dev libxaw7-dev \
    libxrender-dev libxcb-render0-dev libxcb-render-util0-dev \
    gawk \
    python3 python3-pip
```

The last three lines beyond the original toolchain bring in xschem-specific dependencies (`libxrender`, the XCB render libs, and `gawk`) along with Python, which is used by `open_pdks`.

2.2 Build dependencies (Red Hat / Fedora)

```
sudo dnf install -y \
    gcc gcc-c++ make m4 tcsh git \
    tcl-devel tk-devel \
    cairo-devel \
    mesa-libGL-devel mesa-libGLU-devel \
    ncurses-devel \
    libX11-devel libXpm-devel libXt-devel libXaw-devel \
    readline-devel \
    bison flex \
    autoconf automake libtool \
    fftw-devel \
    libXrender-devel \
    gawk \
    python3 python3-pip
```

2.3 Create the install prefix and source directories

Pick the install root once and use it throughout. The default in `setup.sh` is `$HOME/opencircuitdesign`; set it in your shell *first* so the build commands below pick it up:

```
# Match the value in setup.sh (edit setup.sh first if you want a different path)
export OPEN_CIRCUITDESIGN_ROOT="$HOME/opencircuitdesign"

mkdir -p "$OPEN_CIRCUITDESIGN_ROOT"   # source trees clone in here directly
cd "$OPEN_CIRCUITDESIGN_ROOT"
```

2.4 Use the included `setup.sh`

Rather than hand-rolling `PATH`, `LD_LIBRARY_PATH`, `PDK_ROOT`, and the per-tool environment variables, the repository ships a `setup.sh` script that handles all of it. After confirming the `OPEN_CIRCUITDESIGN_ROOT` line at the top of `setup.sh` matches the prefix you chose above, source the script in any shell where you want the tools available:

```
# One-off: source it in the current shell
source /path/to/repo/setup.sh

# Persistent: have every login shell source it automatically
echo 'source /path/to/repo/setup.sh' >> ~/.bashrc
```

The script:

- Verifies `$OPEN_CIRCUITDESIGN_ROOT` exists and bails out loudly if it doesn't.
- Prepends `$OPEN_CIRCUITDESIGN_ROOT/bin` to `PATH` and `$OPEN_CIRCUITDESIGN_ROOT/lib` to `LD_LIBRARY_PATH`.
- Sets `PDK_ROOT` to `$OPEN_CIRCUITDESIGN_ROOT/share/pdk` (the standard `open_pdks` install location when configured with `--with-pdk_root=$OPEN_CIRCUITDESIGN_ROOT/share/pdk`).
- Exports per-tool variables: `MAGIC_HOME`, `NETGEN_HOME`, `NGSPICE_SHARE_DIR`, `XSCHEM_SYSTEM_LIBRARY_PATH`.
- If `sky130A` is found under `$PDK_ROOT`, it also sets `PDK=sky130A`, `SKY130A`, `MAGIC_MAGICRC`, `NETGEN_SETUP`, `XSCHEM_LIBRARY_PATH`, and `XSCHEM_RC`.
- Prints a short summary so you can confirm everything resolved correctly.

You should source `setup.sh` *before* running the verification commands at the end of each section below. The build steps themselves do not need it — they only need `OPEN_CIRCUITDESIGN_ROOT` exported.

3 Magic

Magic is the VLSI layout editor used as the geometry engine for the entire flow.

```
cd "$OPEN_CIRCUITDESIGN_ROOT"
git clone https://github.com/RTimothyEdwards/magic.git
cd magic
./configure --prefix="$OPEN_CIRCUITDESIGN_ROOT"
make -j$(nproc)
make install
```

Verify (after `source setup.sh`):

```
which magic
magic -dnull -noconsole -T scmos <<< "version; quit -noprompt"
```

4 Netgen

Netgen is the LVS (Layout-Versus-Schematic) and netlist comparison tool.

```
cd "$OPEN_CIRCUITDESIGN_ROOT"
git clone https://github.com/RTimothyEdwards/netgen.git
cd netgen
./configure --prefix="$OPEN_CIRCUITDESIGN_ROOT"
make -j$(nproc)
make install
```

Verify:

```
which netgen
netgen -batch <<< "puts [version]; quit"
```

5 IRSIM

IRSIM is a switch-level simulator that integrates with Magic.

```
cd "$OPEN_CIRCUITDESIGN_ROOT"
git clone https://github.com/RTimothyEdwards/irsim.git
cd irsim
./configure --prefix="$OPEN_CIRCUITDESIGN_ROOT" \
            --with-tcl=/usr/lib/tcl8.6 \
            --with-tk=/usr/lib/tk8.6
make -j$(nproc)
make install
```

If your distribution uses a different Tcl/Tk version, adjust the `--with-tcl`/`--with-tk` flags accordingly (or omit them to let `configure` auto-detect).

Verify:

```
which irsim
irsim -V
```

6 ngspice

ngspice is required by `open_pdks` for simulation models and verification. We build it from the official GitHub mirror at https://github.com/ngspice/ngspice rather than the SourceForge tarball, because SourceForge's file layout occasionally shifts and hard-coded version URLs go stale; cloning sidesteps that entirely.

```
cd "$OPEN_CIRCUITDESIGN_ROOT"
git clone https://github.com/ngspice/ngspice.git
cd ngspice
./autogen.sh
mkdir -p release && cd release
../configure --prefix="$OPEN_CIRCUITDESIGN_ROOT" \
             --enable-xspice \
             --enable-cider \
             --enable-openmp \
             --with-readline=yes \
             --disable-debug
```

```
make -j$(nproc)
make install
```

Notes:

- `./autogen.sh` regenerates the `configure` script from `configure.ac`. The git tree does not ship a pre-generated `configure` the way release tarballs do, so this step is mandatory. It needs `autoconf`, `automake`, and `libtool` (already in the prerequisite list).
- To build a tagged release rather than master, run `git tag -l` after cloning to list available tags, then `git checkout <tag>` (e.g. `git checkout ngspice-44`) before running `autogen.sh`.
- `--enable-xspice` is required for code-model support used by many open-source PDKs.
- Building inside a `release/` subdirectory keeps the source tree clean as recommended by the ngspice manual.

Verify:

```
which ngspice
ngspice -v
```

7 xschem

xschem is the schematic capture tool used to drive ngspice simulations and generate netlists for LVS against Magic layouts. It is a separate upstream project from Tim Edwards' tools, but `setup.sh` configures the xschem-related environment variables (`XSCHEM_SYSTEM_LIBRARY_PATH`, `XSCHEM_LIBRARY_PATH`, `XSCHEM_RC`), so we install it into the same prefix.

7.1 Verify xschem build dependencies

If you followed the prerequisite lists in Section 2, you already have everything xschem needs. The xschem-specific packages are:

- **Debian / Ubuntu:** `libxaw7-dev`, `libxrender-dev`, `libxcb-render0-dev`, `libxcb-render-util0-dev`, `tcl-dev`, `tk-dev`, `flex`, `bison`, `libcairo2-dev`, `gawk`.
- **Red Hat / Fedora:** `libXaw-devel`, `libXrender-devel`, `tcl-devel`, `tk-devel`, `flex`, `bison`, `cairo-devel`, `gawk`.

7.2 Clone, configure, build, and install

```
cd "$OPEN_CIRCUITDESIGN_ROOT"
git clone https://github.com/StefanSchippers/xschem.git
cd xschem
./configure --prefix="$OPEN_CIRCUITDESIGN_ROOT"
make -j$(nproc)
make install
```

Notes:

- xschem ships a hand-written `configure` script (not autotools generated). `--prefix` works as expected; most other autotools flags do not apply.

- If you need translucent rectangle / fill rendering, ensure the XCB render packages (`libxcb-render0-dev`, `libxcb-render-util0-dev`) were installed before `configure` ran — otherwise xschem falls back to a non-translucent code path.
- The build installs system symbol libraries to `$OPEN_CIRCUITDESIGN_ROOT/share/xschem/`, which is what `setup.sh` points `XSCHEM_SYSTEM_LIBRARY_PATH` at.

7.3 Verify

```
which xschem
xschem --version

# Launch with no schematic to confirm the GUI starts:
xschem -b
```

After `open_pdks` is installed (next section), `setup.sh` will additionally export `XSCHEM_LIBRARY_PATH` and `XSCHEM_RC` pointing at the sky130A xschem libraries. To open xschem with the sky130A symbol set, run:

```
xschem --rcfile "$XSCHEM_RC"
```

8 open_pdks (install last)

`open_pdks` (https://github.com/RTimothyEdwards/open_pdks) is a meta-installer: it fetches and configures vendor PDKs (sky130A, gf180mcu, etc.) and stitches them into Magic, Netgen, ngspice, and xschem. **It must be the last thing you build.** During `configure` and `make install`, `open_pdks` probes `PATH` for each tool and runs them to generate tech files, magic `.magicrc` files, netgen LVS setup files, and xschem libraries. If any of the tools is missing or points at a different prefix, the resulting PDK install will be incomplete or inconsistent.

Prerequisites for this section

Before running anything in this section, confirm *all* of the following:

1. Magic, Netgen, IRSIM, ngspice, and xschem are all installed under `$OPEN_CIRCUITDESIGN_ROOT` (Sections 3–7 of this guide).
2. You have sourced `setup.sh` in the current shell:

   ```
   source /path/to/repo/setup.sh
   ```

3. All four tools resolve under `$OPEN_CIRCUITDESIGN_ROOT/bin`:

   ```
   which magic netgen ngspice xschem
   ```

   Each path printed should start with the value of `$OPEN_CIRCUITDESIGN_ROOT`. If any of them is missing or points at `/usr/bin`, fix that *first* — do not proceed with `open_pdks`.

8.1 Configure and build

With just `--prefix`, `open_pdks` installs the PDKs under `$OPEN_CIRCUITDESIGN_ROOT/share/pdk`, which is exactly where `setup.sh` points `PDK_ROOT`. Magic, Netgen, and ngspice are auto-detected from `PATH`, so the explicit `--with-magic=`, `--with-netgen=`, and `--with-ngspice=` flags are unnecessary as long as you sourced `setup.sh` (or otherwise put `$OPEN_CIRCUITDESIGN_ROOT/bin` on `PATH`) before running `configure`.

Important: the sky130 and gf180mcu PDKs must be configured and installed *separately* — run `configure`, `make`, and `make install` for one PDK before moving on to the other. Do not combine both `--enable-*-pdk` flags in a single `configure` invocation.

Start by cloning `open_pdks`:

```
cd "$OPEN_CIRCUITDESIGN_ROOT"
git clone https://github.com/RTimothyEdwards/open_pdks.git
cd open_pdks
```

sky130 PDK

The sky130 `configure` pulls in the base sky130 PDK along with the OpenRAM SRAM macros (`--enable-sram-sky130`), the SRAM space variant (`--enable-sram-space-sky130`), the ReRAM macros (`--enable-reram-sky130`), and the OSU standard-cell libraries at the 12T, 15T, and 18T track heights (`--enable-osu-t12-sky130`, `--enable-osu-t15-sky130`, `--enable-osu-t18-sky130`).

```
./configure --prefix="$OPEN_CIRCUITDESIGN_ROOT" \
            --enable-sky130-pdk \
            --enable-sram-sky130 \
            --enable-osu-t12-sky130 \
            --enable-osu-t15-sky130 \
            --enable-osu-t18-sky130 \
            --enable-sram-space-sky130 \
            --enable-reram-sky130
make
make install
```

gf180mcu PDK

Once the sky130 install completes, configure and build the gf180mcu PDK. This pulls in the base gf180mcu PDK along with the OSU standard-cell library (`--enable-osu-sc-gf180mcu`), and the OCD I/O library (`--enable-ocd-io-gf180mcu`).

```
./configure --prefix="$OPEN_CIRCUITDESIGN_ROOT" \
            --enable-gf180mcu-pdk \
            --enable-osu-sc-gf180mcu \
            --enable-ocd-io-gf180mcu
make
make install
```

The `--enable-*-pdk` flags select which PDK to fetch on each pass. Drop any sub-options you don't need; each PDK is several gigabytes after install.

8.2 Install notes (each PDK takes 30–90 minutes)

The PDKs land under `$PDK_ROOT`, e.g. `$OPEN_CIRCUITDESIGN_ROOT/share/pdk/sky130A` and `$OPEN_CIRCUITDESIG` Once `sky130A` exists, sourcing `setup.sh` again will pick it up and export `PDK`, `SKY130A`, `MAGIC_MAGICRC`, `NETGEN_SETUP`, `XSCHEM_LIBRARY_PATH`, and `XSCHEM_RC` automatically.

8.3 Verify

```
source /path/to/repo/setup.sh

ls "$PDK_ROOT"
# Expect: sky130A  sky130B  gf180mcuA  gf180mcuB  gf180mcuD  gf180mcuD  ...

# setup.sh should now report sky130A as the selected PDK:
echo "$PDK"           # -> sky130A
echo "$MAGIC_MAGICRC" # -> .../sky130A/libs.tech/magic/sky130A.magicrc
echo "$NETGEN_SETUP"  # -> .../sky130A/libs.tech/netgen/sky130A_setup.tcl

# Open a sky130 layout in magic to confirm tech files were installed:
magic -rcfile "$MAGIC_MAGICRC"
```

9 Tutorial repository

A companion tutorial for using this tool stack to design analog/digital circuits in the Global Foundries 180 nm MCU process is available at:

https://github.com/stineje/gf180mcu-open-source-vlsi

The repository ships with the `setup.sh` this guide references, schematic and layout examples, simulation testbenches, and step-by-step instructions for the canonical Magic / Netgen / xschem / ngspice flow against the gf180mcu PDK.

9.1 Clone the tutorial

Pick any directory you like for tutorial work — it does *not* need to live under `$OPEN_CIRCUITDESIGN_ROOT`. A common choice is a class or coursework folder:

```
cd ~                  # or wherever you keep coursework
git clone https://github.com/stineje/gf180mcu-open-source-vlsi.git
cd gf180mcu-open-source-vlsi
```

The `ieee_chipathon` branch holds the IEEE ChipAThon materials for this course. The default branch (`main`) tracks upstream development; for coursework, stay on `ieee_chipathon` for now..

9.2 Source the tutorial's `setup.sh`

The tutorial's `instructions-setup/setup.sh` is the same script described in this guide. Source it from your shell so all the tool and PDK environment variables are set:

```
source ./instructions-setup/setup.sh
```

To make this automatic for every login shell, append the line to your `~/.bashrc` with the absolute path:

```
echo "source $HOME/gf180mcu-open-source-vlsi/instructions-setup/setup.sh" \
     >> ~/.bashrc
```

9.3 Verify the tutorial environment

After sourcing, the script prints a summary. Confirm that `PDK = gf180mcuD` (or whatever variant you have installed) and that `magic`, `netgen`, `ngspice`, and `xschem` all resolve under `$OPEN_CIRCUITDESIGN_ROOT/bin`. If `PDK` reports `sky130A` instead of a gf180mcu variant, your gf180mcu install didn't complete — revisit Section 8 before working through the tutorial.

9.4 Keeping up to date

The tutorial is updated periodically with new examples and corrections. To pull the latest on the `ieee_chipathon` branch:

```
cd ~/gf180mcu-open-source-vlsi
git pull
```

10 Putting it all together

10.1 Quick sanity check

After every tool is installed and you have sourced `setup.sh`, the script's own summary output should list each tool's location. You can also run:

```
for tool in magic netgen irsim ngspice xschem; do
    printf "%-8s -> %s\n" "$tool" "$(which $tool 2>/dev/null || echo '<not found>')"
done
```

All paths should resolve under `$OPEN_CIRCUITDESIGN_ROOT/bin`.

10.2 Recommended directory layout after install

```
$OPEN_CIRCUITDESIGN_ROOT
|-- bin/
|   |-- magic
|   |-- netgen
|   |-- irsim
|   |-- ngspice
|   `-- xschem
|-- include/
|-- lib/
|-- share/
|   |-- magic/      # MAGIC_HOME
|   |-- netgen/     # NETGEN_HOME
|   |-- ngspice/    # NGSPICE_SHARE_DIR
|   |-- xschem/     # XSCHEM_SYSTEM_LIBRARY_PATH
|   |-- pdk/        # PDK_ROOT
|   |   |-- sky130A/
```

```
|   |   `-- gf180mcuA/
|   `-- man/
|-- magic/          # cloned source trees (sit alongside bin/, lib/, share/)
|-- netgen/
|-- irsim/
|-- ngspice/
|-- xschem/
`-- open_pdks/
```

11 Troubleshooting

`ERROR: Directory $OPEN_CIRCUITDESIGN_ROOT does not exist`
`setup.sh` aborts when its target prefix is missing. Either create the directory (`mkdir -p "$OPEN_CIRCUITDESIGN_R` or edit the `OPEN_CIRCUITDESIGN_ROOT=...` line at the top of `setup.sh` to point somewhere that does exist.

`configure: error: Tcl headers not found`
Make sure `tcl-dev` / `tcl-devel` is installed and that `tclConfig.sh` lives in a standard location (typically `/usr/lib/tclX.Y/`). Pass `--with-tcl=<dir>` explicitly if needed.

Magic launches but graphics are missing
Build Magic with Cairo and OpenGL support enabled (the default when the corresponding `-dev` packages are present). Re-run `./configure` after installing them and rebuild.

`open_pdks` fails to find Magic / Netgen
Ensure `$OPEN_CIRCUITDESIGN_ROOT/bin` is on `PATH` before running `./configure` (sourcing `setup.sh` does this). If auto-detection still fails, pass absolute paths explicitly via `--with-magic=$OPEN_CIRCUITDESIGN_ROOT` `--with-netgen=$OPEN_CIRCUITDESIGN_ROOT/bin/netgen`, and `--with-ngspice=$OPEN_CIRCUITDESIGN_ROOT/bi`

`setup.sh` warns "`No PDK directory found`"
`open_pdks` hasn't been built/installed yet, or it was configured with a `--prefix` that doesn't match `$OPEN_CIRCUITDESIGN_ROOT`. Re-run `./configure` for `open_pdks` with `--prefix="$OPEN_CIRCUITDESIGN_ROOT"`, or override the PDK location explicitly with `--with-pdk_root="$OPEN_CIRCUITDESIGN_ROOT/share/pdk"`.

Shared libraries not found at runtime
Source `setup.sh` (it sets `LD_LIBRARY_PATH`). For a system-wide alternative, add `$OPEN_CIRCUITDESIGN_ROOT/lib` to `/etc/ld.so.conf.d/opencircuitdesign.conf` and run `sudo ldconfig`.

Updating later
Each tool can be rebuilt independently: `cd $OPEN_CIRCUITDESIGN_ROOT/<tool> && git pull && make && make install`. Re-run `open_pdks` only when changing PDK selection or after a significant Magic/Netgen update.

12 References

- Open Circuit Design home page: http://opencircuitdesign.com
- Magic on GitHub: https://github.com/RTimothyEdwards/magic
- Netgen on GitHub: https://github.com/RTimothyEdwards/netgen
- IRSIM on GitHub: https://github.com/RTimothyEdwards/irsim

- ngspice: https://ngspice.sourceforge.io
- xschem on GitHub: https://github.com/StefanSchippers/xschem
- open_pdks on GitHub: https://github.com/RTimothyEdwards/open_pdks
- gf180mcu open-source VLSI tutorial: https://github.com/stineje/gf180mcu-open-source-vlsi
- IIC-OSIC-TOOLS container: https://github.com/iic-jku/iic-osic-tools

A Container alternative for macOS and Windows

The build-from-source instructions in Sections 3–8 target Linux. On macOS and Windows, native builds of Magic, xschem, and the rest of the stack are painful (or impossible without WSL). The recommended alternative is the **IIC-OSIC-TOOLS** Docker container from the Institute for Integrated Circuits at TU Wien, which bundles Magic, Netgen, IRSIM, xschem, ngspice, KLayout, OpenROAD, yosys, and a pre-installed PDK set (sky130A, gf180mcuD, etc.) into a single image. Repository: https://github.com/iic-jku/iic-osic-tools.

This appendix covers running the container on macOS and Windows. Linux users can also use it, but the from-source path documented earlier is usually preferable on Linux.

A.1 Prerequisites

macOS: Install Docker Desktop from https://www.docker.com/products/docker-desktop/. Launch it once after install so the engine starts; you should see "Engine running" in the lower-left of the Docker Desktop window.

Windows: Install Docker Desktop with the WSL2 backend (the default on Windows 10 and 11). If WSL2 isn't already installed, open PowerShell as administrator and run:

```
wsl --install
```

then reboot. After install, launch Docker Desktop once.

Both platforms: you will run shell commands. On macOS, open the **Terminal** app (Spotlight: `Cmd+Space`, type `Terminal`). On Windows, open the WSL Ubuntu terminal (Start menu) or use Docker Desktop's built-in shell (the `>_` icon at the bottom of the Docker Desktop window). Verify Docker is reachable:

```
docker --version
```

A.2 Apple Silicon caveat

The `hpretl/iic-osic-tools` image is built for x86_64. On Apple Silicon (M1/M2/M3/M4), Docker Desktop runs it under emulation: roughly 2–3× slower than native and uses more RAM, but functional. You may see a warning the first time the image is pulled:

```
WARNING: The requested image's platform (linux/amd64) does not match
         the detected host platform (linux/arm64/v8)
```

That is expected; you can pin the platform explicitly to suppress it:

```
docker pull --platform linux/amd64 hpretl/iic-osic-tools:latest
```

Intel Macs and Windows hosts have no equivalent issue.

A.3 Clone the wrapper scripts and start a VNC session

The container's GUI tools are exposed via a browser-based VNC desktop, which works identically on macOS and Windows and avoids the X11 forwarding problems of XQuartz / VcXsrv. From your terminal:

```
cd ~
git clone https://github.com/iic-jku/iic-osic-tools.git
cd iic-osic-tools
./start_vnc.sh
```

The first run pulls the image (roughly 10 GB — expect 5 to 45 minutes depending on bandwidth). You can watch progress under **Images** in Docker Desktop's left sidebar. When the script finishes starting the container, it prints a URL similar to:

```
http://localhost:8888/vnc.html?autoconnect=true&resize=remote
```

Open that URL in any browser. You will land on a full XFCE desktop with Magic, xschem, KLayout, ngspice, Netgen, and IRSIM all installed. PDKs are at `/foss/pdks/` inside the container — `sky130A`, `gf180mcuD`, etc.

A.4 Mounting your work into the container

By default the container's `/foss/designs` directory is bound to `~/iic-osic-tools-volume/` on the host. Anything saved there from inside the container persists on your machine.

To mount a different host folder — say a clone of the gf180mcu tutorial — pass the path as an argument:

```
./start_vnc.sh ~/work/gf180mcu-open-source-vlsi
```

On Windows from WSL2, refer to Windows-side paths via the `/mnt/c/` mount:

```
./start_vnc.sh /mnt/c/Users/jstine/Documents/work
```

A.5 Using the gf180mcu tutorial inside the container

The PDK layout inside the container matches what `setup.sh` expects, but the container is already pre-configured — you do not need to source `setup.sh`. Inside the VNC desktop, open a terminal and run:

```
cd /foss/designs
git clone https://github.com/stineje/gf180mcu-open-source-vlsi.git
cd gf180mcu-open-source-vlsi
git checkout ieee_chipathon
```

```
export PDK_ROOT=/foss/pdks
export PDK=gf180mcuD
magic -rcfile $PDK_ROOT/$PDK/libs.tech/magic/$PDK.magicrc
```

A.6 Improving the VNC experience

The browser-based VNC is fine for one or two windows. For serious schematic editing, the browser sometimes intercepts keyboard shortcuts. Two fixes:

- Press `F11` in the browser to enter fullscreen mode — this frees most shortcuts.
- Connect with a native VNC client instead. The `start_vnc.sh` script also exposes a raw VNC port (typically `localhost:5901`). On macOS, the built-in **Screen Sharing** app handles `vnc://localhost:5901` cleanly; on Windows, install TigerVNC or RealVNC.

A.7 Stopping, restarting, and updating

The container keeps running until you stop it. From the host terminal:

```
docker ps                       # list running containers
docker stop iic-osic-tools      # stop the container
```

or click the stop button next to the container in the Docker Desktop **Containers** tab. To use it again later:

```
cd ~/iic-osic-tools && ./start_vnc.sh
```

To pull a newer release of the image (the project ships periodic updates):

```
docker pull hpretl/iic-osic-tools:latest
./start_vnc.sh
```

Files in your mounted work directory are preserved across image updates; only the container's read-only OS layer changes.

A.8 When to use this versus the from-source build

The container gets you running in minutes and isolates the entire stack from the host. Trade-offs to keep in mind:

- **Pros:** no native build dependencies; identical environment across macOS / Windows / Linux teammates; one command to update; easy to throw away and recreate.
- **Cons:** 10 GB disk; GUI feels less snappy over VNC than a native window; tool versions follow the container release cadence (you cannot trivially patch Magic and rebuild without forking the image); on Apple Silicon, x86_64 emulation has a measurable performance cost.

For class assignments and tutorial work on macOS or Windows, the container is almost always the right choice. Building from source (Sections 3–8) is preferable when you are hacking on the tools themselves, debugging a specific build issue (such as the gf180mcu `(UNNAMED)` annotation error discussed in Section 11 *Troubleshooting*), or working on Linux with full root access to the host.